3.a Functions

Declaring, Calling & Customizing Kotlin Functions

Contents

1 **Function Declaration & Syntax**
The fun keyword, parameters, return types & calling functions

2 **Default & Named Arguments**
Optional parameters, default values, named argument syntax & overriding defaults

3 **Single-Expression Functions & Unit**
Expression body syntax, return type inference & Unit-returning functions

4 **Variable Arguments & Infix Notation**
vararg parameters, the spread operator & infix function calls

5 **Function Scope & Tail Recursion**
Top-level, local & member functions, generic functions & tailrec optimization

6 **Practice Exercises**
Solved and unsolved exercises covering all function topics

Function Declaration & Syntax

```
fun sum(a: Int, b: Int): Int {
    return a + b
}

// Calling a function
val result = sum(3, 5)  // 8
```

Key Concepts

- Declared with the **fun** keyword
- Parameters use **Pascal notation**: name: Type
- Each parameter must be **explicitly typed**
- Return type after colon: …**): Int**
- Trailing commas allowed in param lists

General Syntax

```
fun functionName(param1: Type1, param2: Type2): ReturnType { /* body */ }
```

Default & Named Arguments

```
// Default argument values
fun read(
    b: ByteArray,
    off: Int = 0,
    len: Int = b.size
) { /*...*/ }
```

```
// Using named arguments
reformat(
    str = "Hello",
    upperCase = false,
    wordSeparator = '_'
)
```

Default Arguments

- Reduce the need for multiple overloads by specifying default values for parameters
- Default values can be expressions (e.g. **len = b.size**), not just constants
- Overriding methods must omit default values from the signature

Named Arguments

- Named arguments can be listed in any order for improved readability
- Positional args must come before named args: **f(1, y = 2)**
- Cannot use named arguments when calling Java functions (names not preserved in bytecode)

Single-Expression Functions & Unit

Single-Expression Functions

```
// With explicit return type
fun double(x: Int): Int = x * 2

// With inferred return type
fun double(x: Int) = x * 2
```

- Use = instead of curly braces for a single expression
- Return type can be inferred by the compiler
- Block-body functions must declare return type explicitly (except Unit)

Unit-Returning Functions

```
// Both are equivalent
fun printHello(name: String?): Unit
fun printHello(name: String?)
```

- Unit = function returns no useful value
- Unit type and return are **both optional**
- Unit is a type with only one value (a singleton)
- You never have to return Unit explicitly

Variable Arguments & Infix Notation

vararg Parameters

```
// fun <... T> arrayOf( vararg elements: T ): Array<T>
// fun <T> listOf(vararg elements: T): List<T>

val array1 = arrayOf(1, 2, 3)  // individual args
println(array1)
println(array1.contentToString())

val list2 = listOf(-1, 0, *array1, 4)  // * spread array1
println(list2)
```

```
Output:
[Ljava.lang.Integer;@452b3a41
[1, 2, 3]
[-1, 0, 1, 2, 3, 4]
```

```
[Ljava.lang.Integer; breaks down as:
[ - indicates it's an array
L - indicates array of objects (not primitives)
java.lang.Integer - the element type
; - type descriptor terminator
```

Infix Functions

```
// infix fun Int.shl(bitCount: Int): Int

 // These are equivalent:
 val x1 = 1 shl 2     // infix
 println(x1)

 val x2 = 1.shl(2)   // regular
 println(x2)
```

```
Output:
4
4
```

Key Rules

- Only **one parameter** may be marked as vararg. Inside the function it appears as an **Array<T>**
- Use the **spread operator *** to pass an existing array to a vararg parameter
- Infix functions must be **member or extension** functions with a **single parameter** (no vararg/default)
- Infix calls have lower precedence than arithmetic operators and type casts

Function Scope & Tail Recursion

Function Scope

- **Top-level** — declared in a file, outside any class (no class wrapper needed like Java)
- **Member functions** — defined inside a class, called with dot notation
- **Local functions** — nested inside other functions, can access outer scope variables (closures)

Generic Functions

- Use angle brackets **<T>** before the function name to specify generic type parameters

- Compiler optimizes recursion into an efficient loop
- Recursive call must be the **last operation**
- Cannot use in try/catch/finally blocks
- Cannot use on **open** functions
- Avoids stack overflow risk

Tail Recursion (tailrec)

```kotlin
tailrec fun findFixPoint(
    x: Double = 1.0
): Double =
    if (x == cos(x)) x
    else findFixPoint(cos(x))
```

```kotlin
fun findFixPoint2(
    x: Double = 1.0
): Double =
    if (x == cos(x)) x
    else findFixPoint2(cos(x))
```

```kotlin
// Test with tailRec - findFixPoint
var fixPoint = 0.0
val time1 = measureTimeMillis {
    for (i in 1..1000_000)
        fixPoint = findFixPoint()
}
println("Fix point: $fixPoint")
println("Time taken TR: $time1")

// Test without tailRec - findFixPoint2
var fixPoint2 = 0.0
val time2 = measureTimeMillis {
    for (i in 1..1000_000)
        fixPoint2 = findFixPoint2()
}
println("Fix point: $fixPoint2")
println("Time taken NO TR: $time2")
```

```
Output:
Fix point: 0.7390851332151607
Time taken TR: 949
Fix point: 0.7390851332151607
Time taken NO TR: 1042
```

Practice Exercises

Practice Exercises

SOLVED

1. Write a function greet(name, greeting) with a default greeting of "Hello"

Use default arguments and call it with/without the greeting parameter.

SOLVED

2. Convert fun max(a: Int, b: Int) to a single-expression function

Practice using = syntax with if-expression body and inferred return type.

UNSOLVED

3. Create a vararg function that returns the average of its arguments

Practice vararg, array operations, and calling with the spread operator.

UNSOLVED

4. Write a tailrec function to compute the nth Fibonacci number

Practice tail recursion with an accumulator pattern to avoid stack overflow.

UNSOLVED

5. Define an infix extension function String.repeat(n) using infix notation

Practice infix functions, extension functions, and calling with infix notation.

Solutions — Problems 1 & 2

Problem 1: greet(name, greeting)

```
fun greet(name: String,
          greeting: String = "Hello") {
    println("$greeting, $name!")
}

greet("Alice")       // Hello, Alice!
greet("Bob", "Hi")   // Hi, Bob!
```

Problem 2: max as single-expression

```
fun max(a: Int, b: Int) =
    if (a > b) a else b

println(max(10, 20)) // 20
println(max(7, 3))   // 7
```

Key Takeaways

- **Problem 1**: Default arguments let callers **omit parameters** with sensible defaults
- **Problem 2**: Single-expression functions use **=** instead of braces; return type is **inferred**
- Kotlin's **if** is an **expression** that returns a value — perfect for single-expression bodies